JUNE 2020

דוח פרויקט-מבוא להנדסת תוכנה

שם המרצה: דוקטור אליעזר גינסבורג

שם הסטודנטית: אסתר הדס וקנין

שנה:תשפ"ה

"MAGICAL UNDERWATER SCENE"
הסצינה שנבנתה

02

תיאור הסצנה
"MAGICAL UNDERWATER SCENE"

מרכיבים עיקריים:

- **מישור מים שקוף**: משמש כרצפת הסצנה (Plane בגוון תכלת עם שקיפות והחזרות).
- **איים**: עשויים מ־Cylinders בצבע חול. עליהם מבנים, צמחייה, גדרות ומדרגות.
- **מבנה מרכזי**: בסיס מ־Cylinder, מגדל נוסף מלמעלה, וגג אדום מ־Sphere.
- **שער וגדרות**: דלת מרכזית מ־Cylinder שחור, גדרות עגולות עשויות Cylinders וספֵרות דקורטיביות עם אור בראש. (פנסים)
- **עצים:** גזע מ־Cylinder ועלווה מספֵרות ירוקות בגדלים שונים.
- **פרחים**: גבעול מ־Cylinder, ראש מ־Sphere, ועלי כותרת מסודרים מספֵרות קטנות.
- **בועות:** ספֵרות שקופות בגובה פני המים ליצירת תחושת זרימה טבעית.
- **דגים**: גוף מרכזי מ־Sphere, ראש נוסף, סנפירים (Triangles), זנב, ועין.
- **פירמידה מזכוכית**: בסיס Polygon עם קירות Triangles מחומר שקוף בגוון כחלחל.
- **תיבה** כהה מתחת לפירמידה: בנויה מ־Polygons, מדמה בסיס מוצק.
- **צינור תת־מימי (Tube)**: זוהר בכחול ומאיר מבפנים.
- **מדרגות**: Polygons המדמים ירידה הדרגתית מהאי למים.
- **עננים צבעוניים:** קבוצה של ספֵרות בגבהים שונים ליצירת רקע תלת־שכבתי.

מקורות תאורה:

- **SpotLight חמים מהצד העליון** – מדמה שקיעת שמש מתחת למים
- **PointLight סביבתי** – תאורה כללית לסצנה
- **SpotLight כחול מתחת למים** – מוסיף אפקט עומק
- **DirectionalLight חם** – אור אלכסוני רך מלמעלה
- **אורות בגדרות** – PointLight בראש כל עמוד
- **אור מתוך הצינור** – SpotLight המדגיש Glow פנימי

הסצנה משלבת צבעים, חומרים, שקיפויות, החזרות ושבירות כדי לייצר עומק, חיות ואווירה טבעית תת־ימית.

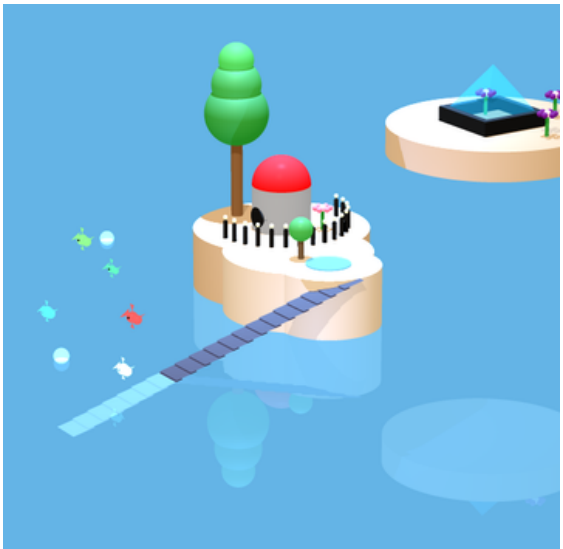
שיפורים שביצעתי

Anti-Aliasing
(דגימה מרובה לפיקסל)

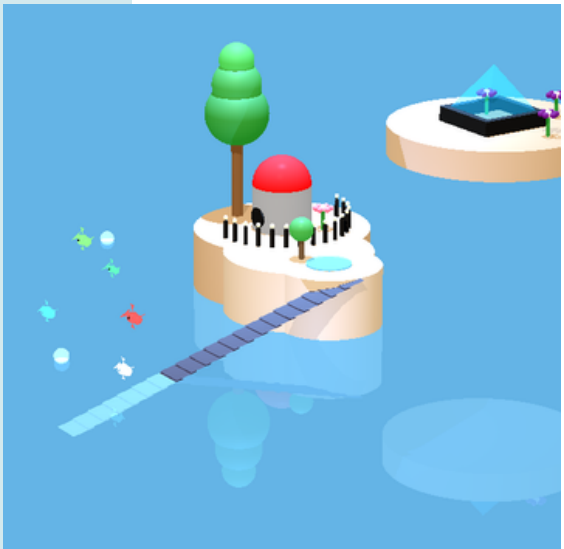
זוהי טכניקת החלקת קצוות, שבמקום לשלוח קרן אחת לכל פיקסל, שולחת רשת צפופה של קרניים (במקרה שלי – 9×9 כלומר 81 קרניים) בתוך שטח הפיקסל. כך, מתקבלת ממוצע של הצבעים, מה שמרכך את המעברים בקצוות אובייקטים, מצמצם זיגזוגים ומונע אפקט "שיניים משוננות" (aliasing) – במיוחד על גופים עם קווים אלכסוניים או מעוגלים. יישמתי זאת על ידי:

- enableAA(true) ו־setAARays(81) במצלמה
- טיפול ב־renderImage כך שיתחשב בריבוי הקרניים ויחשב ממוצע מדויק

אחרי



לפני



04

שיפורים שביצעתי

Adaptive Super Sampling
(דגימה חכמה)

שיפור זה נועד בכדי להימנע מדגימה עודפת כשאין שינוי ויזואלי משמעותי בפיקסל. יצרתי מערכת ששולחת רק 5 קרניים ראשוניות (4 פינות ומרכז), בודקת את הצבעים, ואם יש הבדלים – מתחילה לפצל את הפיקסל לעוד 4 תתי־פיקסלים. התהליך חוזר עד עומק 4 או 5 (לפי setAdaptiveDepth). כך, פיקסלים עם שינוי חד (למשל קצה דג, או ענן מואר) מקבלים פירוט גבוה, ושטחים אחידים (למשל מים) מקבלים רק דגימה אחת. זו דרך חכמה לשמור על איכות גבוהה תוך חיסכון ניכר בזמן.

השיפור ניכר בזמן הריצה

05

שיפורים שביצעתי

(עיבוד מקבילי) *Multithreading*

רינדור תמונה באיכות גבוהה הוא תהליך כבד מאוד – במיוחד עם AA ו־Jitter. לכן מימשתי עבודה מקבילית שבה כל Thread עובד על שורות פיקסלים שונות. השתמשתי ב־RayTracerType.MULTI_THREADED, ומאחוריו יש שימוש ב־ExecutorService, שחילק את העבודה בין הליבות של המעבד. במחשב שלי יש 32 ליבות זמינות, כך שהרינדור ירד מ־~5800 שניות (כמעט שעתיים) ל־~166 שניות בלבד. הקוד עצמו נבדק בטסטים שונים שהשוו בין הרצה רגילה להרצה מקבילית עם אותם פרמטרים.

השיפור ניכר בזמן הריצה

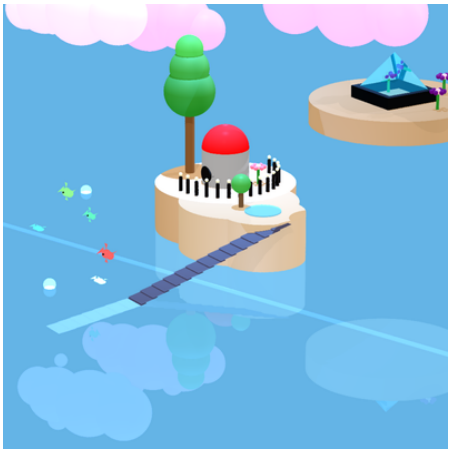
06

בונוסים שביצעתי

(סטייה אקראית) JITTER

Jittering הוא שיפור נוסף מעל Anti-Aliasing: במקום לדגום את הפיקסל בדיוק לפי גריד קבוע, כל קרן מוזזת מעט באופן אקראי (רעש מבוקר) בתוך תחום הפיקסל. זה מדמה תופעות טבעיות של טשטוש, גלים או תנודות. למשל, פני המים נראים נוזליים יותר, הבועות לא חותכות את הגבול בצורה נוקשה, והצללים מקבלים מרקם טבעי. ה־Jitter מופעל בכל אחד מהטסטים באמצעות enableJitter(), וממומש בפונקציית בניית קרניים בתוך constructRayThroughPixel.

אחרי



לפני

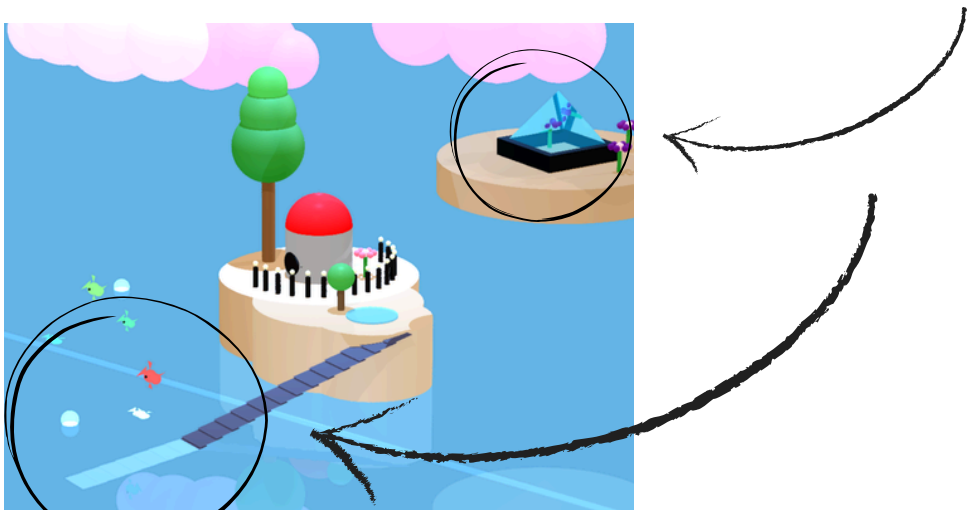
07

בונוסים שביצעתי

שבירת קרניים במים ובזכוכית

מימשתי יכולת של שבירת קרני אור דרך גופים שקופים, כמו מים או זכוכית. לדוגמה:

- הקרניים נשברות כשהן פוגעות במשטח המים (Plane עם חומר בעל KT ו־RefractiveIndex)
- הקרניים נשברות גם בתוך גופים כמו Cylinder שקוף, Tube תת־מימי זוהר, ו־Pyramid זכוכיתית
- השתמשתי ב־createBiasedRay כדי להימנע מקרני השתקפות שגויות ולשפר את יציבות המעבר בין חומרים
- השבירה מתבצעת בפועל על פי חוקי הפיזיקה, כולל זווית פגיעה ושבירה בהתאם לאינדקס שבירה (1.333 למים)



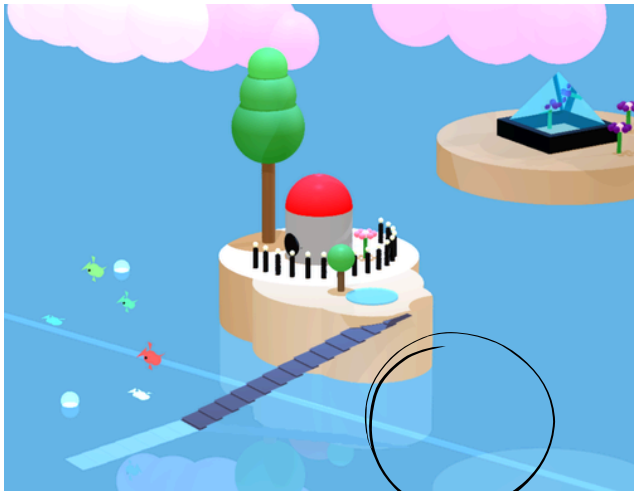
08

בונוסים שביצעתי

שימוש בגוף TUBE (מימוש הפונקציה לקבלת הנורמל)

- יצרתי Tube שקוף וזוהר כחול שמתחת לפני הקרקעית, המדמה זרימת אנרגיה או אור פנימי

והגשת שלב 6 בתוך שבוע



זמני ריצה של השיפורים

הפיצ'ר	זמן (בשניות)
ללא Adaptive וללא Multithreading (רק AA עם Jitter)	5841.24
ללא Adaptive עם Multithreading	2540.41
עם Adaptive (depth = 4) ללא Multithreading	591.00
עם Adaptive (depth = 4) עם Multithreading	166.54

זמני ריצה של השיפורים

מסקנות:

1. **MULTITHREADING בלבד (השוואה בין מקרה 1 למקרה 2)** הביא לקיצור זמן הריצה ביותר מ־50%. השיפור נובע מניצול מלא של ליבות המעבד באמצעות חלוקת העבודה לפעולות מקבילות.
2. **ADAPTIVE SUPER SAMPLING בלבד (השוואה בין מקרה 1 למקרה 3)** תרם לחיסכון משמעותי בכמות הקרניים שנשלחות לפיקסלים, בעיקר באזורים אחידים, והביא להפחתה של כ־90% בזמן הריצה.
3. **השילוב של ADAPTIVE יחד עם MULTITHREADING (מקרה 4)** הציג את זמן הריצה הנמוך ביותר. שילוב זה מצמצם גם את מספר הקרניים הנשלחות וגם מבצע את החישובים במקביל, ולכן מביא ליעילות מקסימלית - הפחתה של כ־97.15% בזמן הריצה לעומת המצב ההתחלתי (ללא ADAPTIVE וללא MULTITHREADING).
4.
5. **ANTI-ALIASING** עם 81 קרניים לפיקסל וללא אופטימיזציה הוא תהליך כבד מאוד, ומהווה את נקודת ההתייחסות לזמן הריצה הגבוה ביותר. מצב זה מדגיש את הצורך בשילוב שיפורים חכמים כדי לקבל תוצאה איכותית בזמן סביר.

11

טכנולוגיה ופיתוח

- **שפת תכנות**: Java
- **סביבת פיתוח:** IntelliJ IDEA